

# 2026-04-11 AIWS × SAM3D × Cadrille 主管汇报

作者: Weld

日期: 2026-04-11

对象: 导师 / 项目主管

定位: 面向汇报的高层总结，重点回答环境、全量实验规模、运行时间、显存/共享内存约束，以及后续如何继续推理。

## 1. 本次工作的核心结论

本轮工作已经把 AIWS5.2 清洗版数据集上的单图 3D 重建流程跑通，并把后续 CAD 生成链路验证到可复用状态：

1. **SAM3D** 环境已经稳定可复现，并在 **RXL** 服务器上完成 1418 个正式样本的全量推理。
2. **SAM3D** 全量运行完成率 **100%**，且具备较完整的单样本时间与显存统计。
3. **Cadrille** 的 **PC** 模态全量推理已跑通，说明 SAM3D STL 输出可以稳定进入 CAD 生成链路。
4. **Cadrille** 的 **IMG** 模态需要额外的共享内存（shm）调优，在默认配置下会因 DataLoader worker 共享内存不足失败，但在修复后已经完成全量成功重试。
5. 后续若继续做批量推理，建议采用“**SAM3D 与 Cadrille 解耦、PC / IMG 分开跑**”的运维策略，而不是直接一次性混合发射。

## 2. 数据集与硬件约束

### 2.1 正式数据口径

- 数据集：**aiws5.2-usable-materialized**
- 正式任务总数：**1418**
- 子集分布：
  - **V1** : 593
  - **V2** : 524
  - **NEW** : 301

### 2.2 数据集整理过程与目录含义

本项目的数据不是一开始就天然按“子集 / 工件 / 输入类型”排好的。原始状态更接近平铺式资源池：

- `aiws5.2-dataset/images/`：所有 RGB 图像基本混放在同一个目录
- `aiws5.2-dataset/depth/`：所有深度文件混放在同一个目录
- `isat_annotations/`：ISAT 标注 JSON，作为最终标注真值来源
- `train.json` / `val.json`：只提供 train / val 归属关系，不适合作为后续实例级重建的唯一真值来源

这种原始组织方式的问题是：

1. 不方便直接按 `V1 / V2 / NEW` 管理
2. 不方便按工件类别批量遍历
3. 多实例样本、无标注样本、异常样本不容易隔离
4. 不利于 SAM3D / Cadrille 这种需要稳定目录约定的批处理流程

因此，后续把数据逐步整理成了 3 层视图：

#### (1) `aiws5.2-usable-split/`

这是保留 **train / val** 划分的非破坏性符号链接视图，结构为：

- `train/`、`val/`
- 每个 split 下再分 `V1/`、`V2/`、`NEW/`
- 每个 subset 下再分 5 类工件：
- `cover_plate`（盖板）
- `square_tube`（方管）
- `h_beam`（H 型钢）
- `channel_steel`（槽钢）
- `bellmouth`（喇叭口）

用途：

- 保留 train / val 语义
- 检查 split 归属是否正确
- 方便后续做按 split 的统计

#### (2) `aiws5.2-usable/`

这是不强调 **train / val**，只强调“可用于正式重建”的清洗视图，结构为：

- 顶层按 `V1 / V2 / NEW`

- 子层按工件类别分组
- 把不适合直接进入主干实验的样本移到 `misc/`

其中：

- `misc/multi_instance/`：一张图中有多个实例
- `misc/multi_label/`：一张图中有多个不同类别
- `misc/unannotated_images/`：有图像但无标注

用途：

- 明确主实验样本边界
- 把干净单实例样本与异常样本隔离

### (3) `aiws5.2-usable-materialized/`

这是最终正式实验使用的数据集根目录，也是 SAM3D 批处理脚本实际读取的目录。

它延续了 `subset / workpiece` 的清晰组织方式，并补充了更适合正式运行的配套文件，例如：

- `metadata/samples.csv`
- `metadata/summary.json`
- `metadata/masks.csv`
- 各工件目录下的 `masks/`

### 正式实验目录的含义

以 `aiws5.2-usable-materialized/` 为例：

- `V1/`：无深度版本
- `V2/`：带 `depth_png/` 的版本
- `NEW/`：带 `depth_exr/` 的版本

每个工件目录下通常包含：

- `images/`：RGB 图像
- `annotations/`：对应 ISAT 标注 JSON
- `depth_png/` 或 `depth_exr/`：深度输入（如果该 subset 有深度）
- `masks/`：由标注 polygon 栅格化生成的实例 mask

辅助目录：

- `metadata/`：机器可读统计与样本清单

- `misc/` : 不进入主实验主干的特殊样本

因此，最终可以把数据组织概括为：

原始平铺资源池 → **split-aware** 视图 → **cleaned usable** 视图 → 正式 **materialized** 实验视图。

## 2.3 运行服务器（RXL）

- CPU: 2 × Intel Xeon Gold 6326 @ 2.90GHz
- CPU 总核数: 64 线程
- 系统内存: 503 GiB
- GPU: 4 × NVIDIA RTX A6000
- 单卡显存: 49140 MiB (约 48 GB)
- 驱动版本: 580.82.09

## 2.4 本轮实验中的关键硬件约束

1. **SAM3D** 主要受 GPU 显存约束，但在 A6000 48 GB 上是可稳定跑通的。
2. **Cadriille PC** 模态对 GPU 调度方式敏感，若多进程错误落到同一张卡上，会触发 CUDA OOM。
3. **Cadriille IMG** 模态的首要瓶颈并不是 GPU 显存，而是 **Docker / PyTorch DataLoader** 的共享内存 (**shm**)。

---

## 3. SAM3D 环境搭建结论

当前正式环境基线如下：

- 项目根目录: `/ssd1/rxl/zhankaiming/AIWS`
- SAM3D 仓库: `/ssd1/rxl/zhankaiming/AIWS/repos/sam-3d-objects`
- Conda 环境: `/home/rxl/anaconda3/envs/sam3d-objects`
- 当前 SAM3D 仓库基线 commit: `81a82373a3a7f4cbb00bd5b32aaf6b4d0f659ddd`

### 3.1 已补齐的关键运行资源

- `checkpoints/hf/pipeline.yaml`
- SAM3D checkpoints
- MoGe 权重
- DINOv2 本地 cache 与 checkpoint

关键路径：

- checkpoints: /ssd1/rxl/zhankaiming/AIWS/repos/sam-3d-objects/checkpoints/hf
- MoGe: /ssd1/rxl/zhankaiming/AIWS/models/moge-vitl/model-real.pt
- DINO cache: /home/rxl/.cache/torch/hub/facebookresearch\_dinov2\_main
- DINO checkpoint:  
/home/rxl/.cache/torch/hub/checkpoints/dinov2\_vitl14\_reg4\_pretrain.pth

### 3.2 性能相关环境结论

为了让 RTX A6000 正常走上更高效的 attention 路径，正式运行使用：

- ATTN\_BACKEND=flash\_attn
- SPARSE\_ATTN\_BACKEND=flash\_attn

实际部署中， flash\_attn 采用了服务器本地源码编译，原因是预编译 wheel 与服务器 glibc/二进制环境不兼容。

更细的安装步骤与命令已整理到《环境搭建与推理 SOP》文档。

## 4. 全量实验汇总

### 4.1 SAM3D 全量实验结果

正式输出目录：

- /ssd1/rxl/zhankaiming/AIWS/outputs/sam3d-aiws52-clean-mesh-stl-20260410-193527

#### 4.1.1 完成情况

| 指标   | 数值   |
|------|------|
| 总任务数 | 1418 |
| 成功数  | 1418 |
| 失败数  | 0    |
| 完成率  | 100% |

4.1.2 总体运行时间

- 最早开始时间： 2026-04-10 19:36:09
- 最晚结束时间： 2026-04-10 21:12:19
- 总墙钟时间：约 1.60 小时
- 按全量墙钟时间折算吞吐：约 884.7 样本 / 小时

分 shard 指标：

| Shard   | GPU | 完成数 | 平均单样本时延 (s) | Shard 吞吐 (样本/小时) |
|---------|-----|-----|-------------|------------------|
| shard-0 | 0   | 355 | 14.074      | 254.337          |
| shard-1 | 1   | 355 | 14.121      | 253.559          |
| shard-2 | 2   | 354 | 14.813      | 241.790          |
| shard-3 | 3   | 354 | 16.191      | 221.272          |

4.1.3 单样本时间统计（1418 样本）

| 指标                | Mean   | P50    | P90    | P95    | Max    |
|-------------------|--------|--------|--------|--------|--------|
| duration_sec      | 14.799 | 13.812 | 17.555 | 18.977 | 73.562 |
| sec_per_megapixel | 7.374  | 6.664  | 8.505  | 9.359  | 51.063 |

4.1.4 显存统计（1418 样本）

| 指标                       | Mean      | P50       | P90       | P95       | Max       |
|--------------------------|-----------|-----------|-----------|-----------|-----------|
| peak_memory_allocated_mb | 18709.201 | 18738.090 | 19428.319 | 19614.863 | 20071.270 |
| peak_memory_reserved_mb  | 24647.275 | 24936.000 | 26420.000 | 27136.000 | 28076.000 |

解释：

- 在 48 GB A6000 上，SAM3D 的峰值 reserved memory 上界约 28.1 GB，说明正式配置存在一定安全余量。
- 从显存角度看，SAM3D 在当前硬件上是可长期复用的，不需要为了全量数据额外换更高显存卡。

4.1.5 运行瓶颈观察

主要长尾来自 V1 / bellmouth 子域，说明后续如果要继续优化总时长，优先分析该类别即可。

## 4.2 Cadrille 全量实验结果

### 4.2.1 PC 模态（全量成功）

用于统计的输出来自一轮“PC 成功、IMG 失败”的全量联合实验；该顶层目录由于后续清理已移入可恢复归档，但 **PC shard** 结果本身完整可用。

统计来源目录：

- `/ssd1/rxl/zhankaiming/AIWS/.trash/outputs-cleanup-20260411-153952/cadrille-full-modalities-20260411-090531-bs64/pc`

配置：

- 模态： `pc`
- 输入：SAM3D 导出的 STL
- `n_samples=5`
- `batch_size=64`
- 4 卡并行
- 选择策略： `evaluate`

结果：

| 指标               | 数值             |
|------------------|----------------|
| 送入 Cadrille 的样本数 | 1418           |
| 成功选出最佳候选数        | 1418           |
| 选择成功率            | 100.0%         |
| 墙钟时间             | 约 80.1 分钟      |
| 折算吞吐             | 约 1061.7 样本/小时 |

补充质量指标（按 shard summary 均值汇总）：

| 指标                         | 数值       |
|----------------------------|----------|
| <code>mean_iou</code> 平均值  | 0.021192 |
| <code>median_cd</code> 平均值 | 0.038003 |

### 4.2.2 IMG 模态（修复后全量成功）

最终保留的成功输出目录：

- `/ssd1/rxl/zhankaiming/AIWS/outputs/cadrille-img-only-20260411-143505-shmfix`

稳定配置：

- 模态：`img`
- `n_samples=1`
- `batch_size=32`
- `--ipc=host --shm-size=16g`
- 降低 IMG dataloader worker 压力
- 选择策略：`evaluate`

结果：

| 指标               | 数值             |
|------------------|----------------|
| 送入 Cadrille 的样本数 | 1418           |
| 成功选出最佳候选数        | 1213           |
| 选择成功率            | 85.54%         |
| 选出的 STL 数        | 1213           |
| 选出的 STEP 数       | 1211           |
| 墙钟时间             | 约 25.9 分钟      |
| 折算吞吐             | 约 3290.3 样本/小时 |

补充质量指标（按 shard summary 均值汇总）：

| 指标                         | 数值       |
|----------------------------|----------|
| <code>mean_iou</code> 平均值  | 0.026166 |
| <code>median_cd</code> 平均值 | 0.047796 |

### 4.2.3 Cadrille 的内存/共享内存约束结论

PC 模态约束：

- 初次多卡全量尝试时，曾出现以下典型错误：



- `torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 20.21 GiB`
- 从日志看，当时多个进程实际挤在同一张 GPU 上，导致单卡剩余显存不足。
- 结论：**PC 模态并非绝对跑不动，而是必须严格 GPU pinning，避免多进程抢同卡。**

#### IMG 模态约束：

- 在 `batch_size=64` 的默认路径下，4 个 shard 均出现：
- `ERROR: Unexpected bus error encountered in worker. This might be caused by insufficient shared memory (shm).`
- `RuntimeError: DataLoader worker ... is killed by signal: Bus error`
- `RuntimeError: unable to write to file </torch_...>: No space left on device (28)`
- 结论：**IMG 模态的首要约束是 Docker/PyTorch DataLoader 的共享内存，而不是 GPU 显存本身。**

#### 4.2.4 为什么目前没有 Cadrille 的精确峰值显存表

当前 Cadrille 运行脚本还没有像 SAM3D 一样记录：

- `torch.cuda.max_memory_allocated()`
- `torch.cuda.max_memory_reserved()`

因此，这一轮对 Cadrille 的“内存总结”更准确地说是：

- 有真实失败证据（OOM / shm bus error）
- 有稳定修复配置
- 但还没有逐样本、逐 shard 的正式峰值显存统计表

这不影响当前结论，但如果后续要写论文或正式答辩材料，建议把这组 instrumentation 加进去。

---

## 5. 面向导师可直接汇报的结论

---

可以直接用下面这组表达：

1. **SAM3D 已在 1418 个正式样本上完成 100% 全量推理**，在 4×RTX A6000 条件下总墙钟时间约 1.6 小时。
2. **SAM3D 单样本平均耗时约 14.8 秒**，峰值 reserved 显存上界约 28.1 GB，说明在 A6000 48 GB 上运行是稳定的。
3. **SAM3D 输出 STL 之后，Cadrille 的 PC 模态已完成全量链路验证**，1418 个输入全部得到可选中的 CAD 结果。

4. **Cadrille** 的 **IMG** 模态不是算力不够，而是共享内存配置不合适；在 `batch=32 + --ipc=host + --shm-size=16g + 降低 worker` 后已跑通全量成功重试。

5. 当前系统已经具备“从 **AIWS** 图像数据，到 **SAM3D 3D** 重建，再到 **Cadrille CAD** 生成”的可复用实验基线。

---

## 6. 下一阶段建议

### 建议 1：把 **Cadrille** 的峰值显存统计补齐

在 `test.py` / `evaluate.py` 外围加入显存记录逻辑，补出：

- 平均显存
- P95 显存
- 最大显存
- 分模态显存表

这样后续汇报会更完整。

### 建议 2：后续批量推理采用“分阶段、分模态”策略

推荐流程：

1. 先单独跑 **SAM3D**，产出 STL
2. 再单独跑 **Cadrille-PC**
3. 最后单独跑 **Cadrille-IMG**

这样可以针对不同模态采用不同 batch size 与 shm 配置，运维风险更低。

### 建议 3：把 **IMG** 失败样本做针对性分析

当前 **IMG** 最终保留率约 `85.5%`，下一步可以重点分析：

- 哪些类别更容易失败
- 是生成失败、转换失败，还是评估选择失败
- 是否与工件几何复杂度、视角、遮挡相关

---

## 7. 附：本次汇报引用的关键输出目录

## 成功保留目录

- SAM3D 正式成功输出：
- `/ssd1/rxl/zhankaiming/AIWS/outputs/sam3d-aiws52-clean-mesh-stl-20260410-193527`
- Cadrille IMG 正式成功输出：
- `/ssd1/rxl/zhankaiming/AIWS/outputs/cadrille-img-only-20260411-143505-shmfix`

## 归档但仍可恢复的统计目录

- Cadrille PC 全量成功统计目录：
- `/ssd1/rxl/zhankaiming/AIWS/.trash/outputs-cleanup-20260411-153952/cadrille-full-modalities-20260411-090531-bs64/pc`

---

## 8. 一句话总结

在 4×RTX A6000 条件下，SAM3D 已经具备全量稳定重建能力，Cadrille 链路也已经验证可行；真正需要精细调优的重点不再是“能不能跑”，而是“如何在不同模态下更稳、更可测地跑”。